SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.* *(BL2, BL3)*

Assessment Tool Weightage: 35%

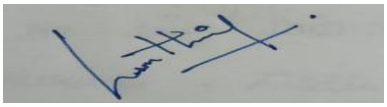
Dr.J.Velmurugan / Professor
Dept. of CSE

QUESTIONS: 1 **Total Marks:100**

Annexure A

Concept Mapping

Code of Conduct:
I Hasmath Nisha (192524513) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Total
Concept Identification	25%					
Linkages	25%					
Organization	20%					
Integration of Literature	20%					
Clarity	10%					
Total						

Annexure A

Concept Mapping

Question 1
Construct a multi-level concept map integrating:

Operating System Types → OS Structures → System Calls → OS Functions.

Your map must show cross-links (e.g., how OS structure influences system call implementation).

Question 2
Design a comprehensive concept map representing:
Process Management, Memory Management, File System, and I/O System.
Include interdependencies and indicate where system calls interact with each component.

Question 3
Create a comparative concept map for:
Batch OS, Time-Sharing OS, Distributed OS, and Real-Time OS.
Highlight differences, similarities, advantages, and limitations using linking phrases.

Question 4
Develop a dynamic concept map showing the execution flow:

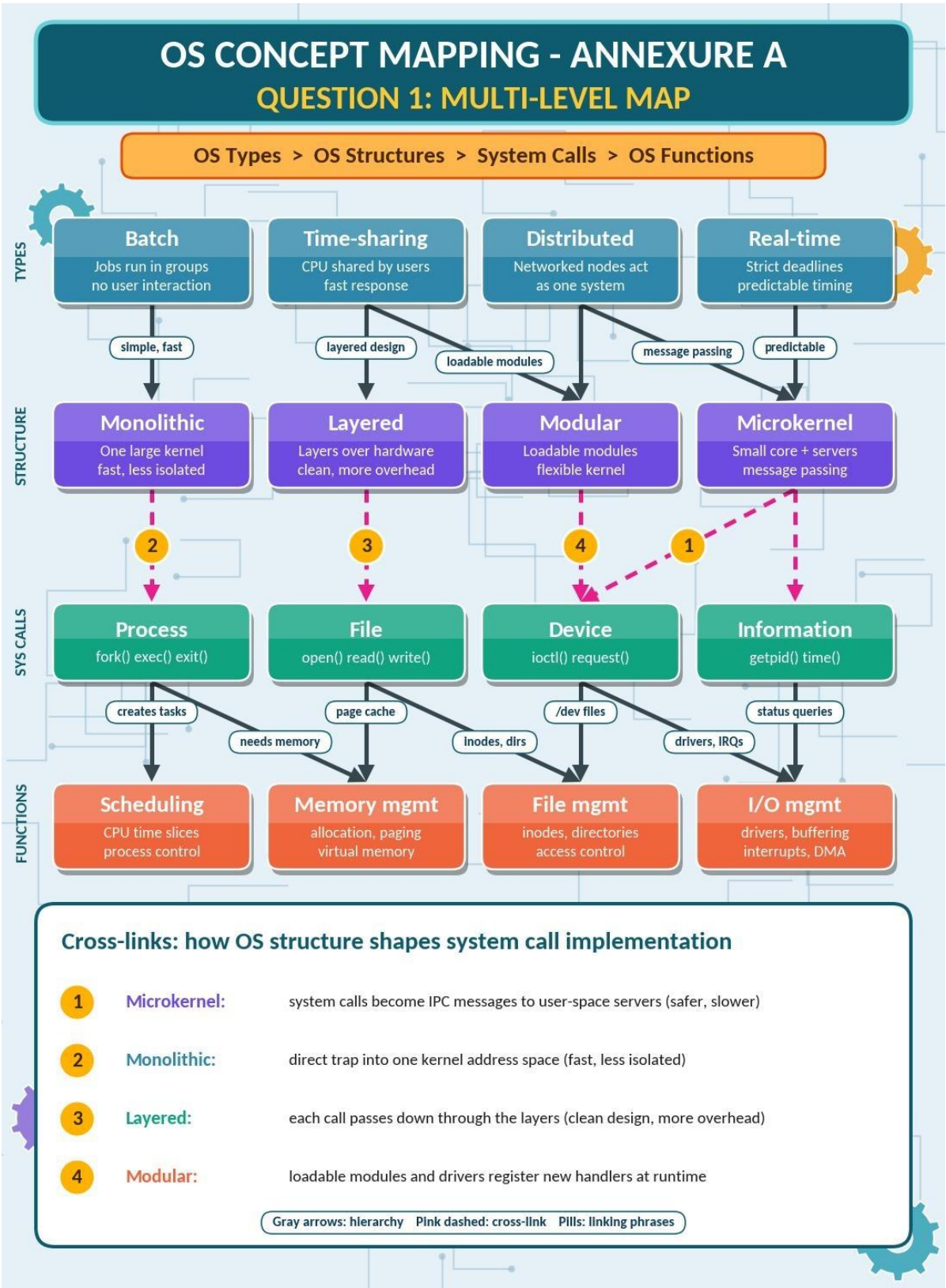
User Program → API → System Call Interface → Kernel → Hardware → Response back to User.

Include feedback loops and control flow paths.

Question 5
Construct a hierarchical + functional concept map for system calls:
Process, File, Device, and Information system calls.
Also show how they relate to OS services like scheduling, memory allocation, and device handling.

OS CONCEPT MAPPING - ANNEXURE A

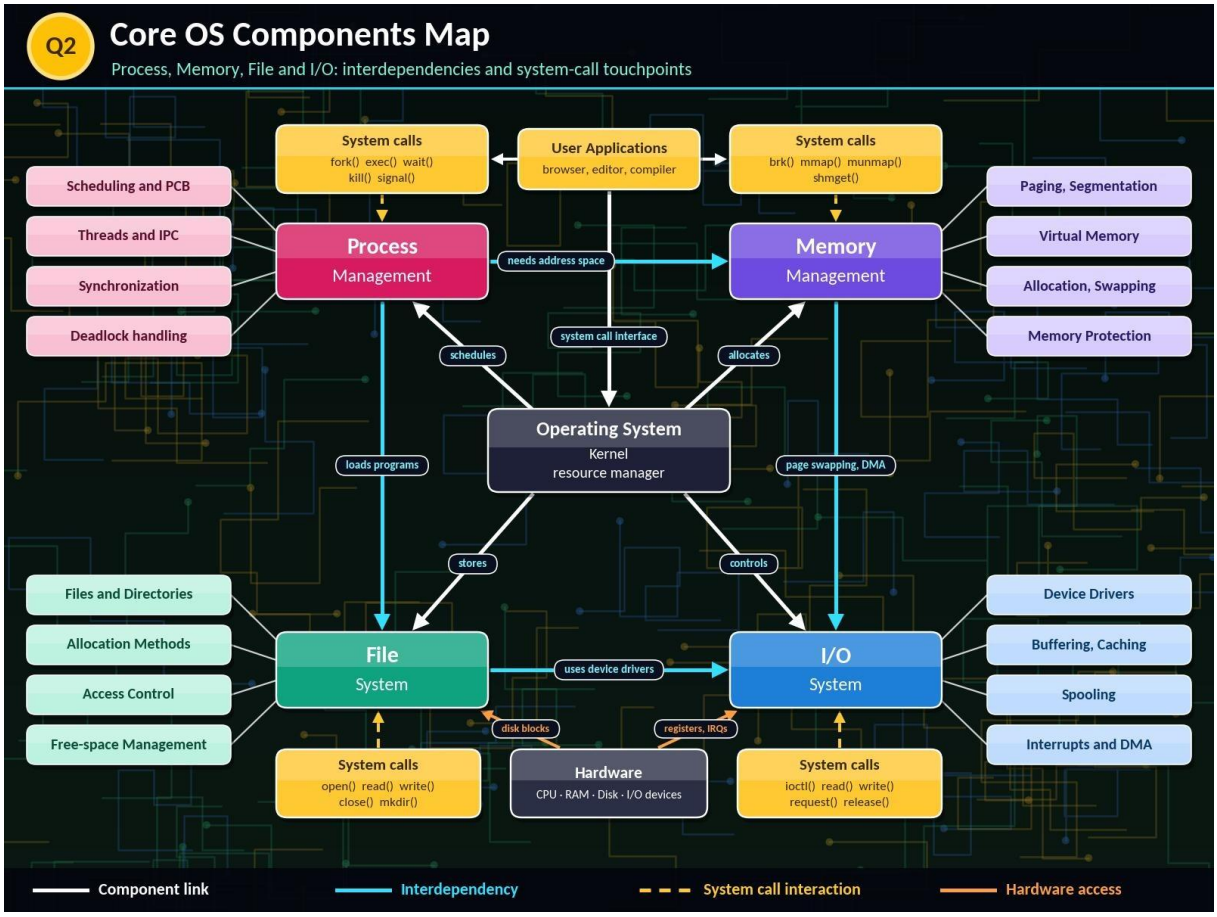
QUESTION 1: MULTI-LEVEL MAP

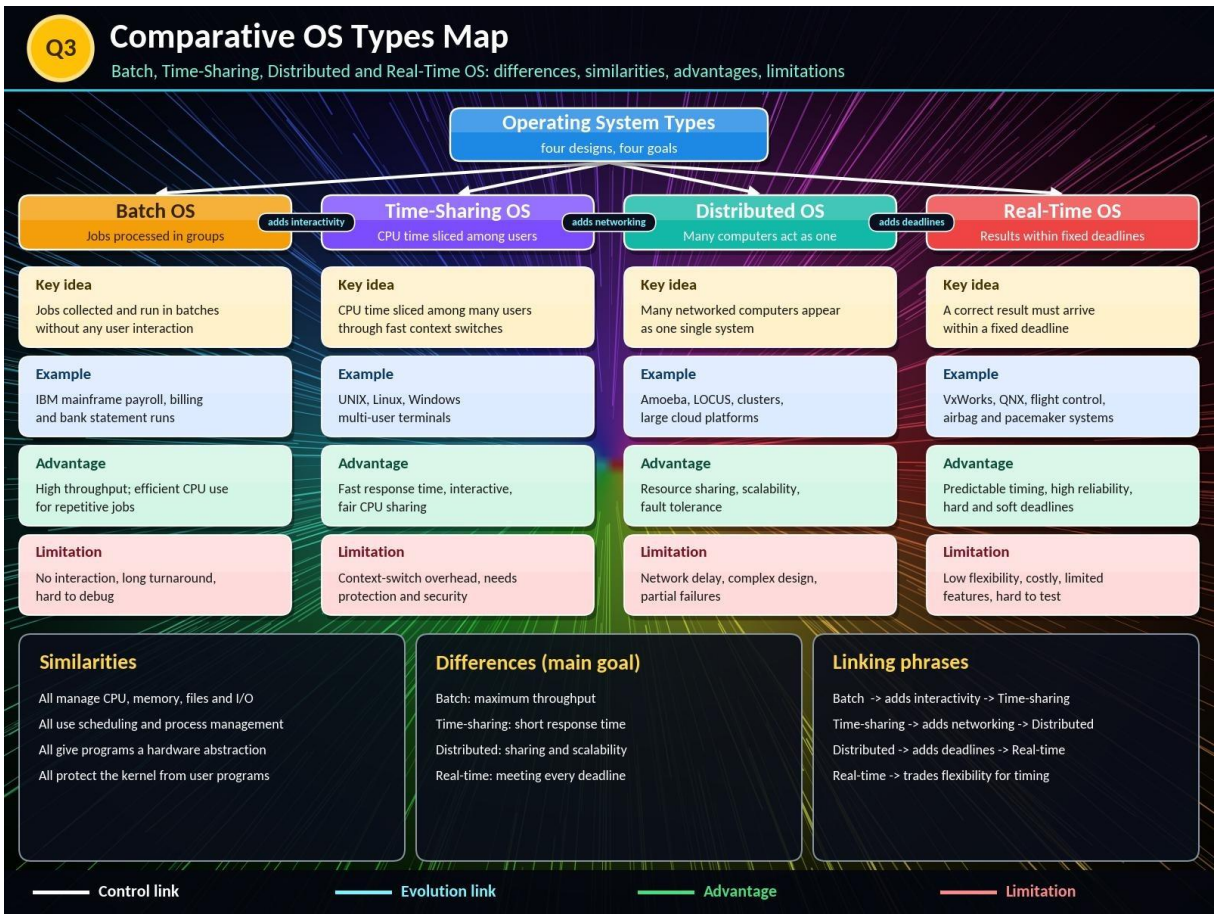


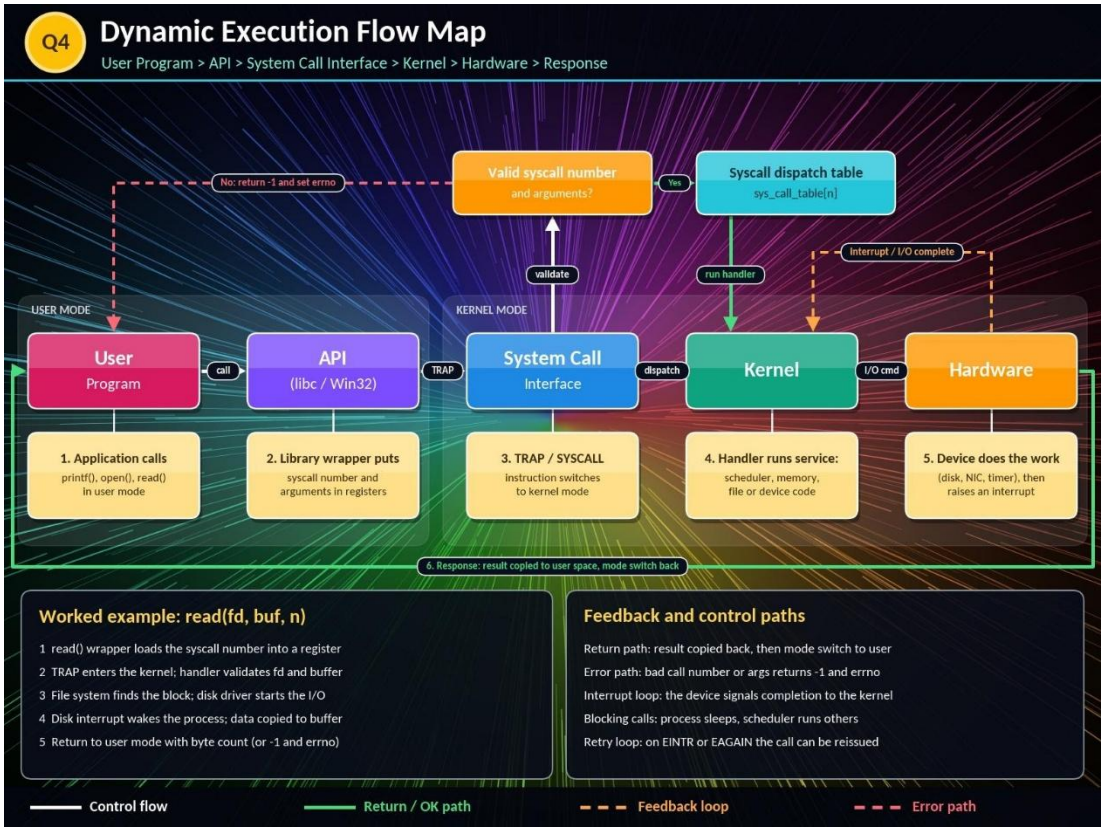
Cross-links: how OS structure shapes system call implementation

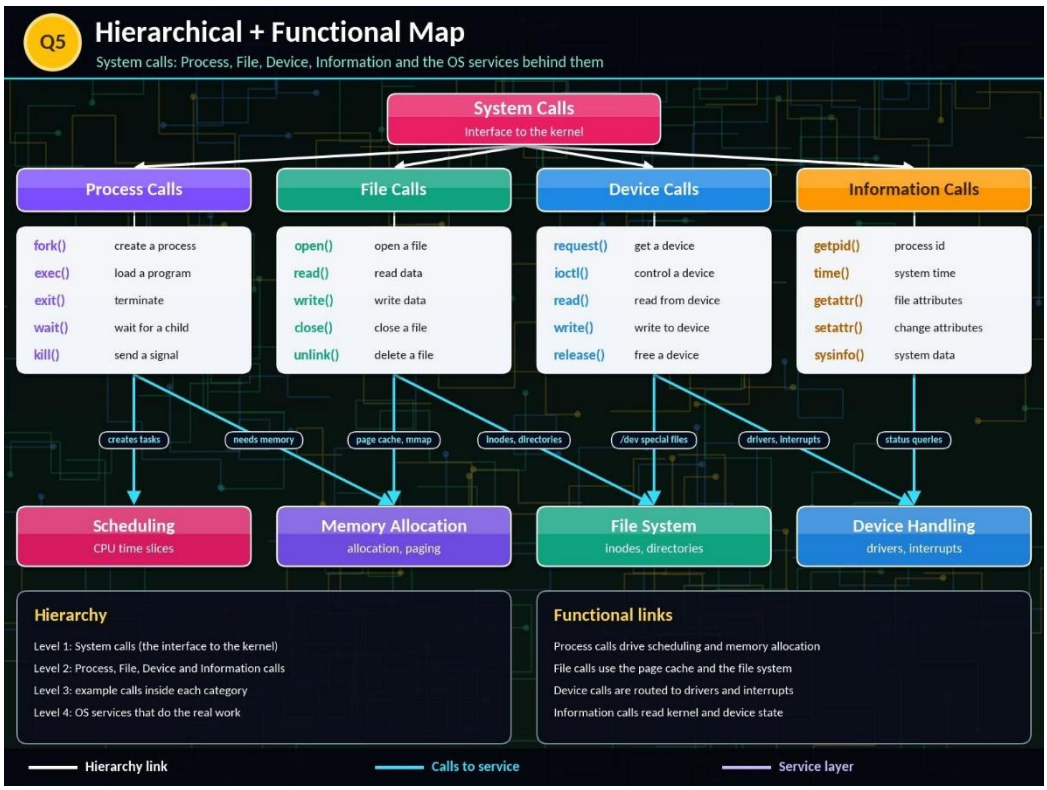
1. **Microkernel:** system calls become IPC messages to user-space servers (safer, slower)
2. **Monolithic:** direct trap into one kernel address space (fast, less isolated)
3. **Layered:** each call passes down through the layers (clean design, more overhead)
4. **Modular:** loadable modules and drivers register new handlers at runtime

Gray arrows: hierarchy Pink dashed: cross-link Pills: linking phrases


Q2
Core OS Components Map
Process, Memory, File and I/O: interdependencies and system-call touchpoints
System calls
fork() exec() wait()
kill() signal()
User Applications
browser, editor, compiler
System calls
brk() mmap() munmap()
shmget()
Scheduling and PCB
Threads and IPC
Synchronization
Deadlock handling
Process
Management
needs address space
Memory
Management
Paging, Segmentation
Virtual Memory
Allocation, Swapping
Memory Protection
schedules
system call interface
allocates
Operating System
Kernel
resource manager
loads programs
page swapping, DMA
stores
controls
Files and Directories
Allocation Methods
Access Control
Free-space Management
File
System
uses device drivers
I/O
System
Device Drivers
Buffering, Caching
Spooling
Interrupts and DMA
disk blocks
registers, IRQs
System calls
open() read() write()
close() mkdir()
Hardware
CPU · RAM · Disk · I/O devices
System calls
ioctl() read() write()
request() release()
Component link
Interdependency
System call interaction
Hardware access


Q3
Comparative OS Types Map
Batch, Time-Sharing, Distributed and Real-Time OS: differences, similarities, advantages, limitations
Operating System Types
four designs, four goals
Batch OS
Jobs processed in groups
adds interactivity
Time-Sharing OS
CPU time sliced among users
adds networking
Distributed OS
Many computers act as one
adds deadlines
Real-Time OS
Results within fixed deadlines
Key idea
Jobs collected and run in batches without any user interaction
Key idea
CPU time sliced among many users through fast context switches
Key idea
Many networked computers appear as one single system
Key idea
A correct result must arrive within a fixed deadline
Example
IBM mainframe payroll, billing and bank statement runs
Example
UNIX, Linux, Windows multi-user terminals
Example
Amoeba, LOCUS, clusters, large cloud platforms
Example
VxWorks, QNX, flight control, airbag and pacemaker systems
Advantage
High throughput; efficient CPU use for repetitive jobs
Advantage
Fast response time, interactive, fair CPU sharing
Advantage
Resource sharing, scalability, fault tolerance
Advantage
Predictable timing, high reliability, hard and soft deadlines
Limitation
No interaction, long turnaround, hard to debug
Limitation
Context-switch overhead, needs protection and security
Limitation
Network delay, complex design, partial failures
Limitation
Low flexibility, costly, limited features, hard to test
Similarities
All manage CPU, memory, files and I/O
All use scheduling and process management
All give programs a hardware abstraction
All protect the kernel from user programs
Differences (main goal)
Batch: maximum throughput
Time-sharing: short response time
Distributed: sharing and scalability
Real-time: meeting every deadline
Linking phrases
Batch -> adds interactivity -> Time-sharing
Time-sharing -> adds networking -> Distributed
Distributed -> adds deadlines -> Real-time
Real-time -> trades flexibility for timing
Control link
Evolution link
Advantage
Limitation


Q4
Dynamic Execution Flow Map
User Program > API > System Call Interface > Kernel > Hardware > Response
Valid syscall number
and arguments?
No: return -1 and set errno
Yes
Syscall dispatch table
sys_call_table[n]
validate
run handler
Interrupt / I/O complete
USER MODE
KERNEL MODE
User
Program
call
API
(libc / Win32)
TRAP
System Call
Interface
dispatch
Kernel
I/O cmd
Hardware
1. Application calls
printf(), open(), read()
in user mode
2. Library wrapper puts
syscall number and
arguments in registers
3. TRAP / SYSCALL
instruction switches
to kernel mode
4. Handler runs service:
scheduler, memory,
file or device code
5. Device does the work
(disk, NIC, timer), then
raises an interrupt
6. Response: result copied to user space, mode switch back
Worked example: read(fd, buf, n)
1 read() wrapper loads the syscall number into a register
2 TRAP enters the kernel; handler validates fd and buffer
3 File system finds the block; disk driver starts the I/O
4 Disk interrupt wakes the process; data copied to buffer
5 Return to user mode with byte count (or -1 and errno)
Feedback and control paths
Return path: result copied back, then mode switch to user
Error path: bad call number or args returns -1 and errno
Interrupt loop: the device signals completion to the kernel
Blocking calls: process sleeps, scheduler runs others
Retry loop: on EINTR or EAGAIN the call can be reissued
Control flow
Return / OK path
Feedback loop
Error path


Q5
Hierarchical + Functional Map
System calls: Process, File, Device, Information and the OS services behind them
System Calls
Interface to the kernel
Process Calls
fork() create a process
exec() load a program
exit() terminate
wait() wait for a child
kill() send a signal
File Calls
open() open a file
read() read data
write() write data
close() close a file
unlink() delete a file
Device Calls
request() get a device
ioctl() control a device
read() read from device
write() write to device
release() free a device
Information Calls
getpid() process id
time() system time
getattr() file attributes
setattr() change attributes
sysinfo() system data
creates tasks
needs memory
page cache, mmap
inodes, directories
/dev special files
drivers, interrupts
status queries
Scheduling
CPU time slices
Memory Allocation
allocation, paging
File System
inodes, directories
Device Handling
drivers, interrupts
Hierarchy
Level 1: System calls (the interface to the kernel)
Level 2: Process, File, Device and Information calls
Level 3: example calls inside each category
Level 4: OS services that do the real work
Functional links
Process calls drive scheduling and memory allocation
File calls use the page cache and the file system
Device calls are routed to drivers and interrupts
Information calls read kernel and device state
Hierarchy link
Calls to service
Service layer

RUBRICS FOR CALCULATION

Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand